SCIENCEUTSAV®
Science is awesome

I'm a ROBO SCIENTIST

Touch-Based Smart Control Systems

Session 7

TOUCH-BASED SMART CONTROL SYSTEMS

Curiosity Kickoff

- What happens when you touch a touch sensor?
- Which is faster: a touch sensor or a push button?
- Why are touch sensors used in modern devices?

Concept Connection

Today, let's meet three special kinds of lights hiding inside your kit. Think of them as the "eyes" of you

This topic introduces students to Touch-Based Smart Control Systems. The goal is to keep the learning simple, practical, and beginner-friendly. Students first understand the concept in easy words, then see a small Arduino program, and then connect the code to the real output or input behavior.

1**.1 Watt LED PCB**

1 Watt LED is a high-brightness light-emitting diode that produces much stronger light compared to normal small LEDs.

2.DIP LED (Dual In-line Package LED)

- A DIP LED is a traditional through-hole light-emitting diode designed with two leads for easy mounting on printed circuit boards. It provides reliable and efficient illumination for indicators, displays, decorative lighting, and electronic projects, and is available in various colors, sizes, and brightness levels.

3.Touch Sensor

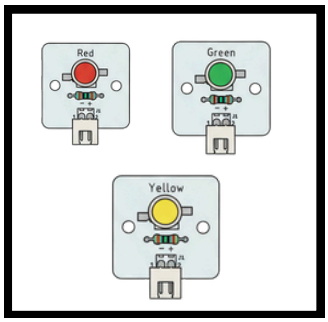
A touch sensor is an electronic input device that detects touch or proximity from a finger without requiring mechanical pressure. It provides a simple and reliable way to control electronic circuits, making it ideal for switches, automation systems, consumer electronics, and interactive projects.

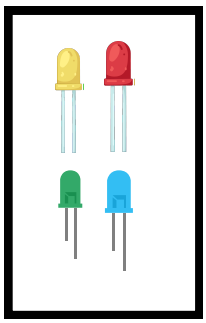
4.Converter

A converter is an electronic module that changes electrical power from one voltage level, current type, or signal format to another. It enables compatibility between different devices and power sources, making it widely used in power supplies, battery-operated systems, industrial equipment, and electronic projects.

5.String LED

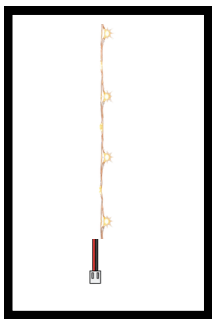
A String LED is a series of multiple LEDs connected along a flexible wire or cable to provide decorative and ambient lighting. It offers energy-efficient illumination and is widely used for home décor, festive decorations, events, signage, and creative lighting projects.











Beginner: Touch sensor Basics

Class Explanation

In this class, students learn the basic idea of Touch sensor Basics. The code below is kept simple so beginners can focus on understanding the main concept without extra complexity.

Materials Required

- Arduino with attached shield
- 1 × Touch sensor
- 1 × Multicolour LED
- RMC wires
- USB cable

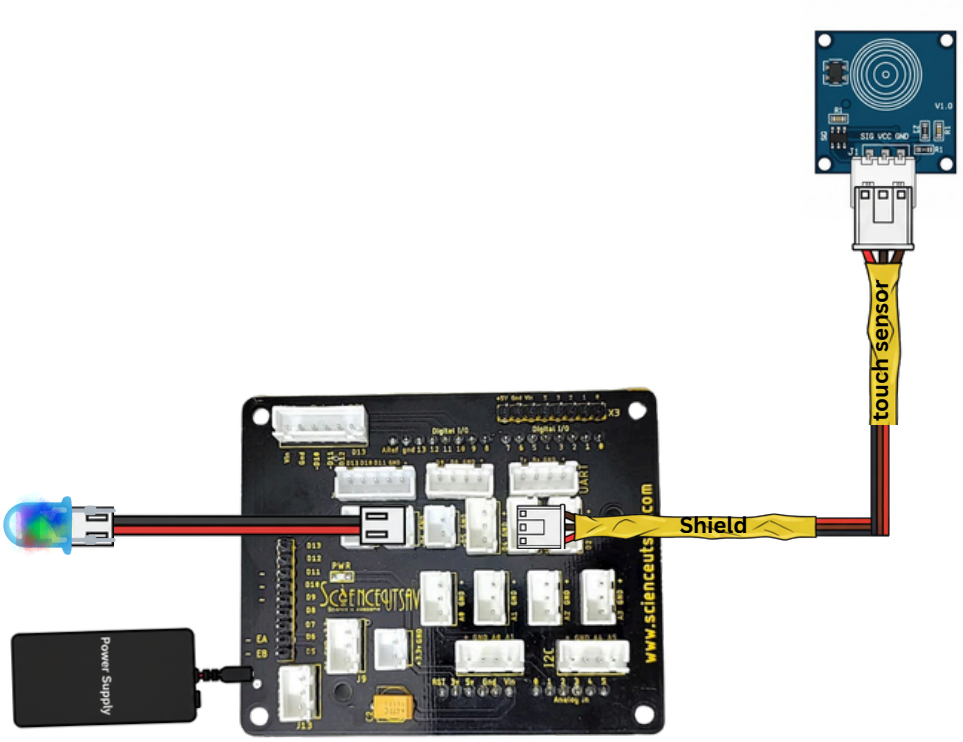
How to Connect

Use RMC wires to connect :

- 1 × Touch sensor → Arduino pin D3 and GND
- 1 × Multicolour LED → Arduino pin D6 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
const int touchSensor = 3;
const int dipLed = 6;

void setup() {
 pinMode(touchSensor, INPUT);
 pinMode(dipLed, OUTPUT);
}

void loop() {
 if (digitalRead(touchSensor) == HIGH) digitalWrite(dipLed, HIGH);
 else digitalWrite(dipLed, LOW);
}
```

Code Explanation

- In this Arduino program, a touch sensor is connected to pin 3 (touchSensor) and an LED is connected to pin 6 (dipLed).
- The setup() function runs once when the Arduino starts. It sets the touch sensor as INPUT and the LED as OUTPUT using pinMode().
- The loop() function runs continuously. It reads the touch sensor value using digitalRead().
- If the touch sensor is activated (HIGH signal), the LED turns ON using digitalWrite(HIGH). If the sensor is not touched (LOW signal), the LED turns OFF using digitalWrite(LOW).
- So overall, the LED glows when the touch sensor is touched and turns OFF when it is released.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Touch sensor Basics but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
const int touchSensor = 3;
const int dipLed1 = 6;
const int dipLed2 = 7;

void setup() {
  pinMode(touchSensor, INPUT);
  pinMode(dipLed1, OUTPUT);
  pinMode(dipLed2, OUTPUT);
}

void loop() {
  if (digitalRead(touchSensor) == HIGH) {
    digitalWrite(dipLed1, HIGH);
    digitalWrite(dipLed2, HIGH);
  } else {
    digitalWrite(dipLed1, LOW);
    digitalWrite(dipLed2, LOW);
  }
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn

the basic idea of Touch sensor Basics how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Intermediate: Touch Sensor with LED

Class Explanation

In this class, students learn the basic idea of Touch Sensor with LED. The code below is kept simple so beginners can focus on understanding the main concept without extra complexity.

Materials Required

- Arduino with attached shield
- 1 × Touch Sensor
- 1 × Red 1W SMD LED on breakout board
- RMC wires
- USB cable

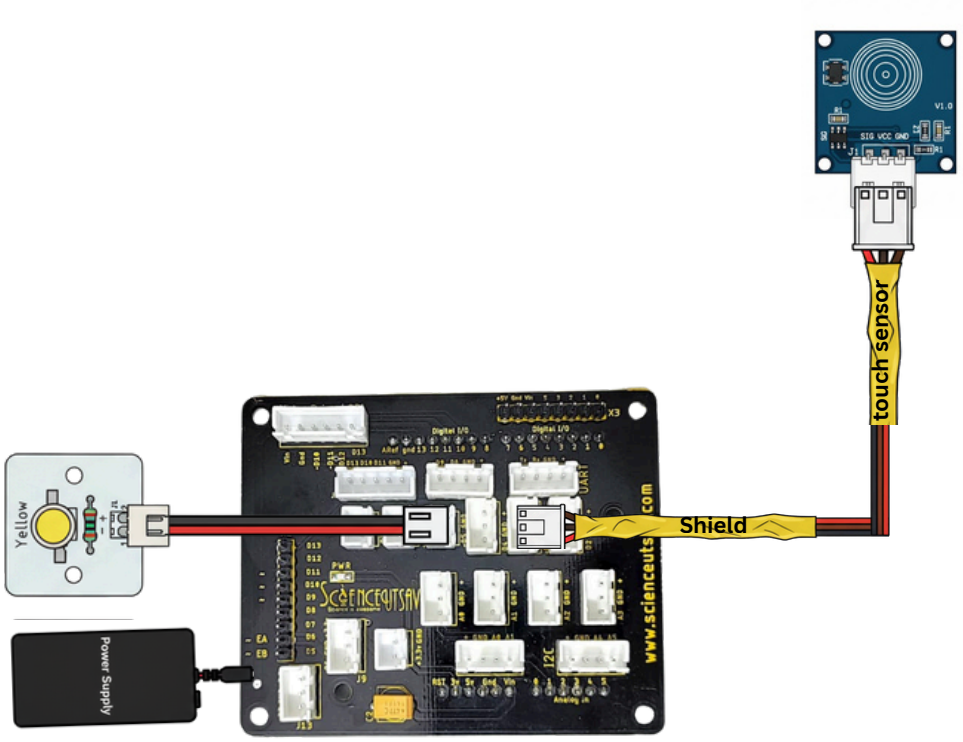
How to Connect

Use RMC wires to connect SMD LED breakout board directly to the Arduino:

- Touch sensor → Arduino pin D2 and GND
- Yellow SMD LED → Arduino pin D5 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
const int touchSensor = 2;
const int led = 5;

void setup() {
  pinMode(touchSensor, INPUT);
  pinMode(led, OUTPUT);
}

void loop() {
  if (digitalRead(touchSensor) == HIGH) {
    digitalWrite(led, HIGH);   // LED ON
  } else {
    digitalWrite(led, LOW);    // LED OFF
  }
}
```

Code Explanation

- In this Arduino program, a touch sensor is connected to pin 2 (touchSensor) and an LED is connected to pin 5 (led).
- The setup() function runs once when the Arduino starts. It sets the touch sensor as INPUT and the LED as OUTPUT using pinMode().
- The loop() function runs continuously. It reads the touch sensor value using digitalRead().
- If the touch sensor is activated (HIGH signal), the LED turns ON using digitalWrite(HIGH). If the touch sensor is not activated (LOW signal), the LED turns OFF using digitalWrite(LOW).
- So overall, the LED glows when the touch sensor is touched and turns OFF when it is released.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Touch Sensor with LED but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
const int touchSensor = 2;
const int led1 = 5;
const int led2 = 6;
const int led3 = 7;

void setup() {
  pinMode(touchSensor, INPUT);

  pinMode(led1, OUTPUT);
  pinMode(led2, OUTPUT);
  pinMode(led3, OUTPUT);
}

void loop() {
  if (digitalRead(touchSensor) == HIGH) {
    digitalWrite(led1, HIGH);
    digitalWrite(led2, HIGH);
    digitalWrite(led3, HIGH);
  } else {
    digitalWrite(led1, LOW);
    digitalWrite(led2, LOW);
    digitalWrite(led3, LOW);
  }
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:

the basic idea of Touch Sensor with LED how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Advanced: Touch Sensor With Led String

Class Explanation

In this class, students learn the basic idea of Touch Sensor With Led String. The code below is kept simple so beginners can focus on understanding the main concept without extra complexity.

Materials Required

- Arduino with attached shield
- 1 × Touch sensor
- 1 × String LED
- RMC wires
- USB cable

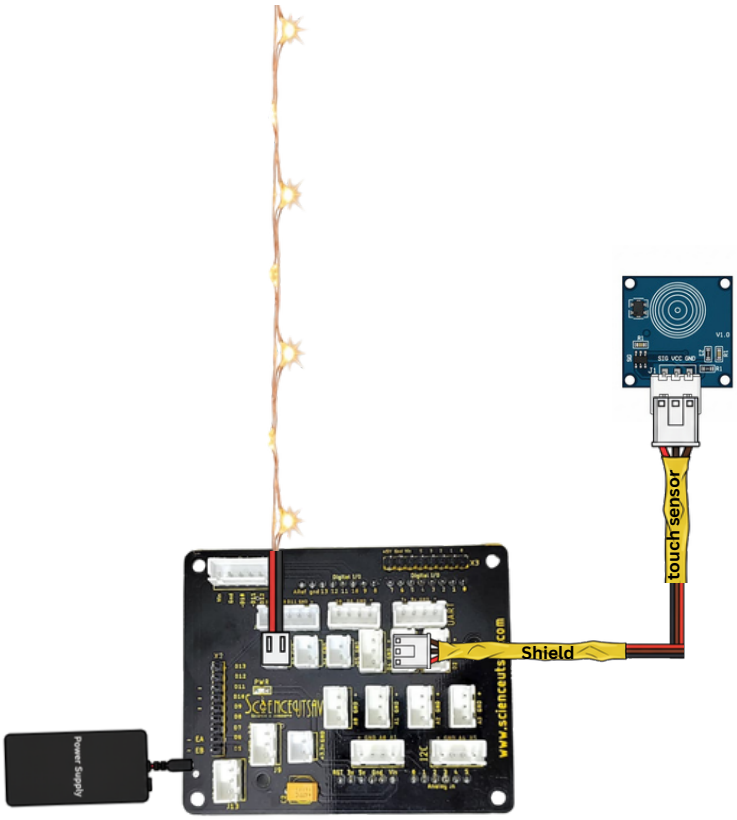
How to Connect

Use RMC wires to connect SMD LED breakout board directly to the Arduino:

- 1 × Touch Sensor→ Arduino pin D3 and GND
- 1 × String led → Arduino pin D7 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
const int touchSensor = 3;
const int stringLed = 7;

int pattern = 0;
bool lastTouchState = LOW;

void setup() {
 pinMode(touchSensor, INPUT);
 pinMode(stringLed, OUTPUT);
}

void loop() {
 bool touchState = digitalRead(touchSensor);

 // Change pattern on every new touch
 if (touchState == HIGH && lastTouchState == LOW) {
  pattern++;

  if (pattern > 3) {
   pattern = 0;
  }

  delay(300);
 }

lastTouchState = touchState;

 // Pattern 0: LED OFF
 if (pattern == 0) {
  analogWrite(stringLed, 0);
 }
```

```
 // Pattern 1: Fade in and fade out
 if (pattern == 1) {
  for (int brightness = 0; brightness <= 255; brightness += 5) {
   analogWrite(stringLed, brightness);
   delay(20);
  }

  for (int brightness = 255; brightness >= 0; brightness -= 5) {
   analogWrite(stringLed, brightness);
   delay(20);
  }
 }

 // Pattern 2: Slow blinking
 if (pattern == 2) {
  analogWrite(stringLed, 255);
  delay(500);
  analogWrite(stringLed, 0);
  delay(500);
 }

 // Pattern 3: Fast blinking
 if (pattern == 3) {
  analogWrite(stringLed, 255);
  delay(100);
  analogWrite(stringLed, 0);
  delay(100);
 }
}
```

Code Explanation

- In this Arduino program, a touch sensor is connected to pin 5 (touchSensor) and an LED is connected to pin 7 (stringLed).

- The setup() function runs once when the Arduino starts. It sets the touch sensor as INPUT and the LED as OUTPUT using pinMode().

- The loop() function runs continuously. It reads the touch sensor using digitalRead() and stores the previous state in lastTouchState. Each time the sensor is touched (LOW to HIGH transition), the pattern value increases. After pattern 3, it resets back to pattern 0.

- The program uses if conditions to create different LED patterns based on the value of pattern:

- Pattern 0: LED remains OFF.
- Pattern 1: The LED smoothly fades ON and OFF using analogWrite().
- Pattern 2: The LED blinks slowly with a 500 ms delay.
- Pattern 3: The LED blinks quickly with a 100 ms delay.

- A small delay() is added after each touch to avoid multiple pattern changes from a single touch.

- So overall, each touch changes the LED to the next lighting pattern, allowing one touch sensor to control multiple LED effects.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Touch Sensor With Led String but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
const int touchSensor = 2;
const int stringLed = 5;
const int dipLed = 6;

void setup() {
  pinMode(touchSensor, INPUT);
  pinMode(stringLed, OUTPUT);
  pinMode(dipLed, OUTPUT);
}

void loop() {
  if (digitalRead(touchSensor) == HIGH) {
    digitalWrite(stringLed, HIGH);
    digitalWrite(dipLed, HIGH);
  } else {
    digitalWrite(stringLed, LOW);
    digitalWrite(dipLed, LOW);
  }
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:
the basic idea of Touch Sensor With Led String how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Expert: Touch Sensor With Multiple LEDs

Class Explanation

In this class, students learn the basic idea of Touch Sensor With Multiple LEDs. The code below is kept simple so beginners can focus on understanding the main concept without extra complexity.

Materials Required

- Arduino with attached shield
- 1 × Touch sensor
- 1 × Yellow 1W SMD LED on breakout board
- 2 × Green 1W SMD LED on breakout board
- 1 × Red 1W SMD LED on breakout board
- Converter
- RMC wires
- USB cable

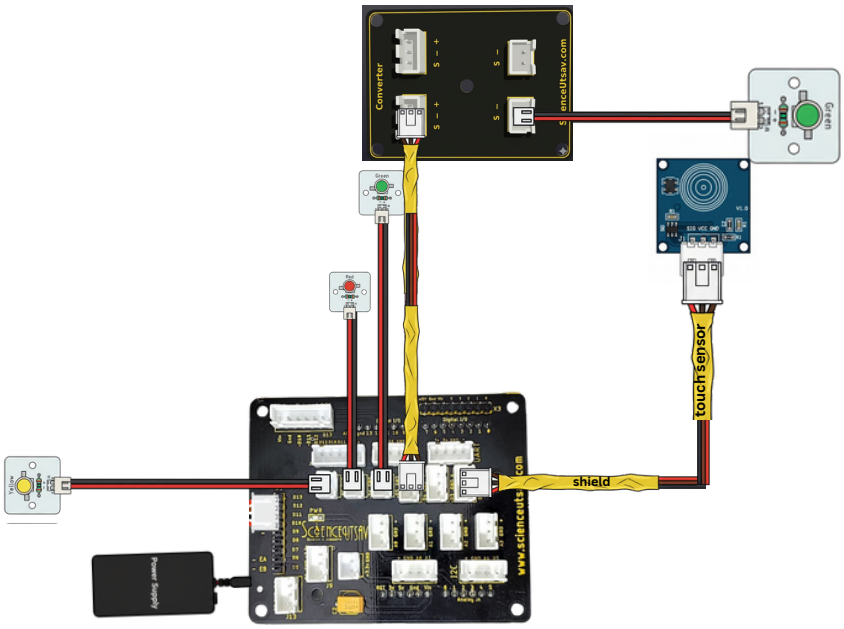
How to Connect

Use RMC wires to connect SMD LED breakout board directly to the Arduino:

- 1 × Touch Sensor → Arduino pin A0 and GND
- 1 × Yellow 1W SMD LED → Arduino pin D2 and GND
- 1 × Red 1W SMD LED → Arduino pin D6 and GND
- 1 × Green 1W SMD LED → Arduino pin D5 and GND
- Converter→ Arduino pin D4 and GND
- 1 × Green 1W SMD LED → Connected to converter

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
const int touchSensor = 2;
const int leds[] = {4, 5, 6, 7};

int touchCount = 0;
bool lastTouchState = LOW;

void setup() {
 pinMode(touchSensor, INPUT);

 for (int i = 0; i < 4; i++) {
 pinMode(leds[i], OUTPUT);
 }
}

void allLeds(bool state) {
 for (int i = 0; i < 4; i++) {
 digitalWrite(leds[i], state);
 }
}

void loop() {
 bool touchState = digitalRead(touchSensor);

 if (touchState == HIGH && lastTouchState == LOW) {
 touchCount = (touchCount + 1) % 5;
 delay(300);
 }
 lastTouchState = touchState;

 switch (touchCount) {

 case 0: // All LEDs OFF
 allLeds(LOW);
 break;

 case 1: // All LEDs ON
 allLeds(HIGH);
 break;
```

```
 case 2: // One by one blinking
 for (int i = 0; i < 4; i++) {
 allLeds(LOW);
 digitalWrite(leds[i], HIGH);
 delay(200);
 }
 break;

 case 3: // Alternate LEDs
 digitalWrite(leds[0], HIGH);
 digitalWrite(leds[1], LOW);
 digitalWrite(leds[2], HIGH);
 digitalWrite(leds[3], LOW);
 delay(300);

 digitalWrite(leds[0], LOW);
 digitalWrite(leds[1], HIGH);
 digitalWrite(leds[2], LOW);
 digitalWrite(leds[3], HIGH);
 delay(300);
 break;

 case 4: // Fast blinking
 allLeds(HIGH);
 delay(150);
 allLeds(LOW);
 delay(150);
 break;
 }
}
```

Code Explanation

- In this Arduino program, a touch sensor is connected to pin 2 (touchSensor) and four LEDs are connected to pins 4, 5, 6, and 7 using an array (leds[]).
- The setup() function runs once when the Arduino starts. It sets the touch sensor as INPUT and all four LED pins as OUTPUT using pinMode().
- A separate function called allLeds() is used to turn all four LEDs ON or OFF together.

- The loop() function runs continuously. It reads the touch sensor using digitalRead(). Each time the sensor is touched (LOW to HIGH transition), the touchCount increases and cycles through five different modes.

- Depending on the value of touchCount, a switch-case statement controls the LED patterns:

- Mode 0: All LEDs OFF.
- Mode 1: All LEDs ON.
- Mode 2: LEDs blink one by one in sequence.
- Mode 3: Alternate LEDs blink in a pattern.
- Mode 4: All LEDs blink rapidly together.

- A small delay() is added to prevent multiple detections from a single touch.

- So overall, each touch changes the LED display pattern, allowing one touch sensor to control multiple lighting effects.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Touch Sensor With Multiple LEDs but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
const int touchSensor = 2;

const int dipLed1 = 4;
const int dipLed2 = 5;
const int dipLed3 = 6;
const int dipLed4 = 7;

const int leds[] = {dipLed1, dipLed2, dipLed3, dipLed4};

int touchCount = 0;
bool lastTouchState = LOW;

void setup() {
  pinMode(touchSensor, INPUT);

  pinMode(dipLed1, OUTPUT);
  pinMode(dipLed2, OUTPUT);
  pinMode(dipLed3, OUTPUT);
  pinMode(dipLed4, OUTPUT);
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:

the basic idea of Touch Sensor With Multiple LEDs how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Learning Outcomes

After completing this session, students should be able to:

- the basic idea of touch sensors and learn how they are used as digital input devices in Arduino projects.
- the basic idea of touch sensor with Arduino and control an LED using simple programming logic.
- the basic idea of one input can control multiple outputs and learn the basics of synchronized lighting systems.
- the basic Learn to program multiple LED outputs using a single touch input and understand interactive lighting control.

Try Out Other Things

- use the Try This Yourself code after each main class code before asking students to build a completely new variation
- ask students to point out the exact lines that changed between the first code and the Try This Yourself code
- let students first run the Try This Yourself code code as it is and then change one value, one condition, or one output